# Software Requirements Specification

**for**

# FridgeFriend

**Version 4.1 approved**

**Prepared by:**

**PM:** Willa Galipeau-Eldridge
**Designer:** Paige Doucette
**Developers:** Jennifer Morin, Drew Hafford, and Isabel Layland
**Team Development**

**02/20/2026**

*Copyright © 1999 by Karl E. Wiegers. Permission is granted to use, modify, and distribute this document.*

# Table of Contents

**Table of Contents** ii
**Revision History** ii
**1. Introduction** 1
1.1 Purpose 1
1.2 Document Conventions 1
1.3 Intended Audience and Reading Suggestions 1
1.4 Product Scope 1
1.5 References 1
**2. Overall Description** 2
2.1 Product Perspective 2
2.2 Product Functions 2
2.3 User Classes and Characteristics 2
2.4 Operating Environment 2
2.5 Design and Implementation Constraints 2
2.6 User Documentation 2
2.7 Assumptions and Dependencies 3
**3. External Interface Requirements** 3
3.1 User Interfaces 3
3.2 Hardware Interfaces 3
3.3 Software Interfaces 3
3.4 Communications Interfaces 3
**4. System Features** 4
4.1 Item Management 4
4.2 Cost Tracking 4
4.3 Expiration Calendar 4
4.4 Account 4
**5. Other Nonfunctional Requirements** 5
5.1 Performance Requirements 5
5.2 Safety Requirements 5
5.3 Security Requirements 5
5.4 Software Quality Attributes 5
5.5 Business Rules 5
**6. Other Requirements** 5
**Appendix A: Glossary** 5
**Appendix B: Analysis Models** 5
**Appendix C: To Be Determined List** 6

# Revision History

| Name | Date | Reason For Changes | Version |
|---|---|---|---|
| Initial Version | 02/20/2026 | Initial | 1.1 |
| Requirements | 02/26/2026 | Added Functional and Non-Functional Requirements to Sections 4 and 5 | 1.2 |
| UI Interface | 02/26/2026 | Added UI Wireframes to Section 3 | 1.3 |
| Introduction | 03/09/2026 | Completed Section 1 (Introduction) | 2.0 |
| Overall Description | 03/10/2026 | Completed Section 2 (Overall Description) | 2.1 |
| External Interface Requirements | 03/11/2026 | Completed Section 3 | 2.2 |
| UI Mockups | 03/13/2026 | Added link to updated UI Mockups | 2.3 |
| Requirements | 03/29/2026 | Revised Functional and Non-Functional Requirements based on Troy's feedback | 3.1 |
| Requirements | 03/29/2026 | Added stimulus and response sequences for Functional Requirements | 3.2 |
| External interface Requirements | 4/15/26 | Added firebase to the software interfaces | 4.1 |

# 1. Introduction

## 1.1 Purpose

The application FridgeFriend is an inventory management system that is designed to help college students/households track food items, reduce duplicate purchases, and promote better household spending. The main objective of this application is to provide college students a way to reliably track their food inventory in a quick and easy way. This document specifies the software requirements that are needed to create the application.

## 1.2 Document Conventions

This document was created based on the IEEE template for System Requirements Specification documents.

The following conventions in the document were used as follows:

| Convention | Description |
|---|---|
| FridgeFriend | App Name: FridgeFriend |
| HTTP | Hypertext Transfer Protocol |
| FREQ | Functional Requirement |
| NFREQ | Non-Functional Requirement |

## 1.3 Intended Audience and Reading Suggestions

This document will be made for developers, project managers, marketing staff, users, testers, and documentation writers. Developers should focus on Section 3 (External Interface Requirements) and Section 4 (System Features). Designers should direct their attention to Section 3.1 (User Interfaces). Testers should use Section 5 (Other Nonfunctional Requirements) to verify the performance of FridgeFriend and ensure security benchmarks are met. The rest of this SRS contains an introduction to the application, external interface requirements, system features, non-functional requirements, and other requirements.

## 1.4 Product Scope

FridgeFriend is a food inventory management software that allows users to see their physical inventory through a digital method of tracking their food supplies.. This software aims to allow users to maintain an accurate inventory of their fridge, freezer, and pantry contents by adding,

editing, and deleting items. By providing sight into physical inventory, the application aims to reduce duplicate purchases, minimize household waste, and promote better spending habits.

### 1.5 References

| Title | Source |
|---|---|
| FridgeFriend GitHub Repository | https://github.com/COS420-S2025/TeamD.git |
| Project Description | FridgeFriend: Project Description |
| Use Case Diagram/Descriptions | FridgeFriend: Use Case Diagrams/Descriptions |

## 2. Overall Description

### 2.1 Product Perspective

FridgeFriend is an alternative to prior solutions to track food inventory, focusing on college students by making sure the application isn't too complex. It serves as an inventory tool for single users or a multiple person household to capture your real-time food inventory. While the application is a standalone-system it utilizes cloud-based synchronization to ensure that inventory data is consistent across many linked devices within a single household.

### 2.2 Product Functions

**Login Page**
- *Welcomes User*
- *Allows users to login*

**Homepage**
- *Links to all other pages*
- *Shows user notifications*

**Inventory Page**
- *Shows list of food inventory*
- *Shows user quantity of item*
- *Allows users to add items*
- *Allows users to scan a barcode*
- *Allows users to edit items*
- *Allows users to delete items*

**Cost Tracking Page**
- *Shows monthly expenses*
- *Shows yearly expenses*
- *Shows wasted money calculation*

**Expiration Calendar Page**
- *Shows user monthly calendar*
- *Allows user to change months*
- *Allows user to click on item for more information*
- *Shows user expiration date for item*

**Household Page**
- *Shows user household members*
- *Allows users to add household member*
- *Allows users to remove a household member*

## 2.3 User Classes and Characteristics

The primary user consists of college students and budget-conscious households who need a quick and easy way to monitor their food supplies. These users are expected to utilize the application frequently to keep an accurate inventory, requiring an intuitive interface for fast item entry. Another user class includes household members that share a synchronized account to allow them to coordinate their purchases and collectively reduce waste. Project managers, developers, and testers serve as internal stakeholders to utilize this documentation to ensure the application meets performance and security requirements.

## 2.4 Operating Environment

FridgeFriend is developed to be a mobile application using a React App with HTML, CSS, and TypeScript. FridgeFriend will operate on all major operating systems including IOS and Android.

## 2.5 Design and Implementation Constraints

The development is constrained to the use of React, HTML, CSS, and TypeScript for the application's front-end. Developers must implement specific security measures, including data encryption and two-factor authentication for every login to protect users personal information from third-parties. Furthermore, FridgeFriend must adhere to the specific regulatory standards of California food labeling laws to facilitate food safety.

## 2.6 User Documentation

Users will be provided with an in-app "Help" section and a digital user manual that details how for users to sync household accounts and use core features.

## 2.7 Assumptions and Dependencies

FridgeFriend is developed using React, with a TypeScript library that incorporates HTML and CSS along with TypeScript to render the web pages. If a user is blocking or disabled TypeScript from their browser the application won't function properly.

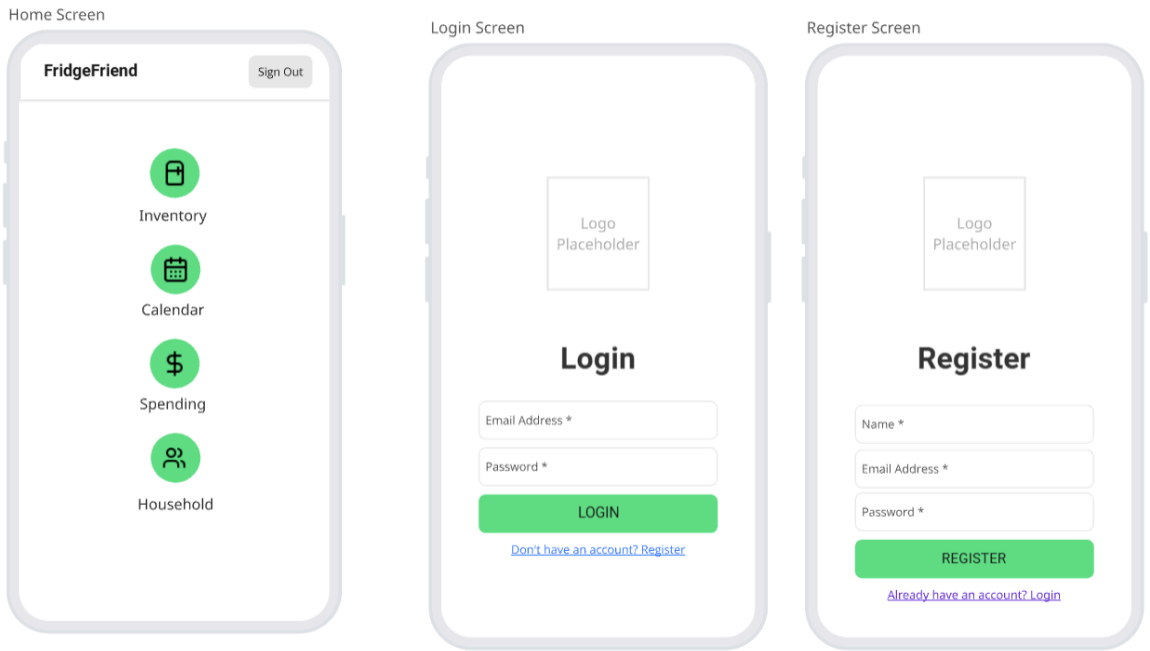
# 3. External Interface Requirements

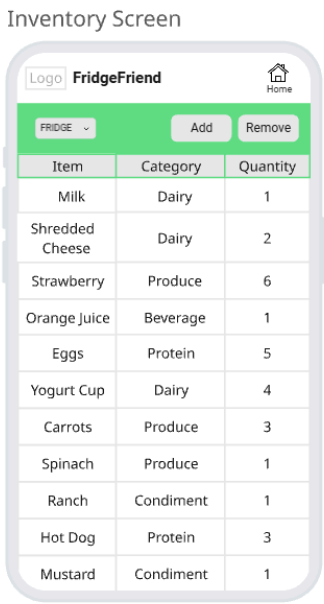
## 3.1 User Interfaces

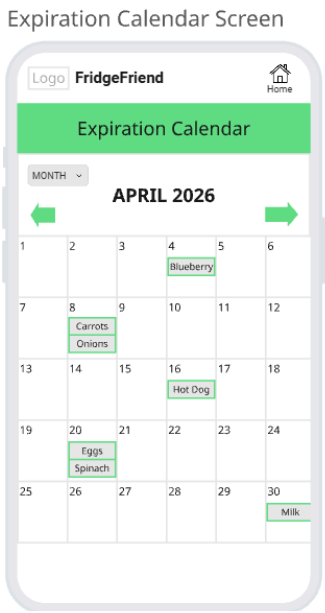
The user interface is designed for mobile application efficiency, with interfaces for food inventory, expense tracking, expiration calendar, and household management. There will be a homepage with links allowing users to navigate through to the various interfaces.

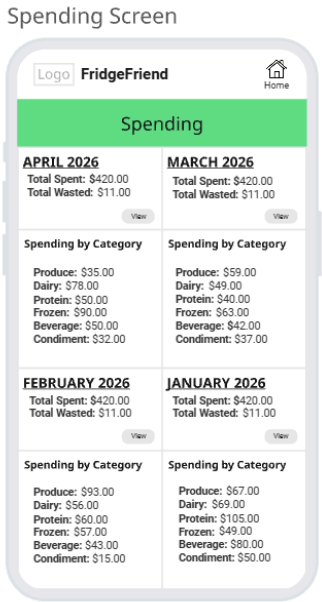
## FridgeFriend: UI Mockups

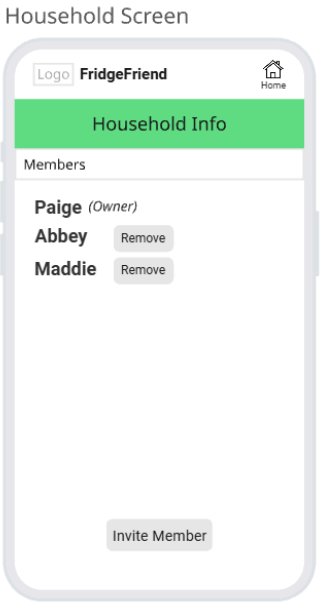
The login interface gives the user the option to login to an existing account or gives the option to create a new account by entering your username, email, and a password. After logging in, FridgeFriend will bring users to the homepage to interact with the rest of the application interfaces. From all pages, users will be able go back and forth through the interfaces and return to the homepage.





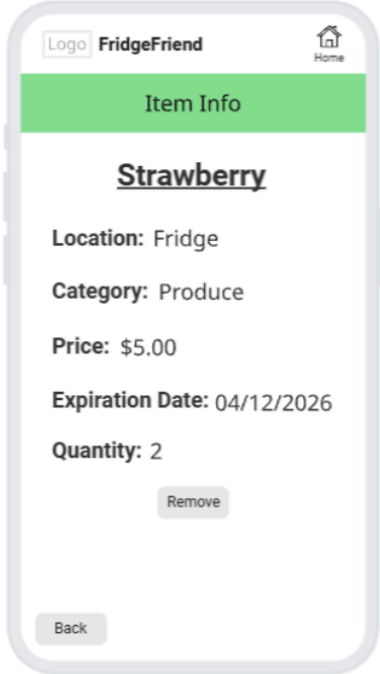



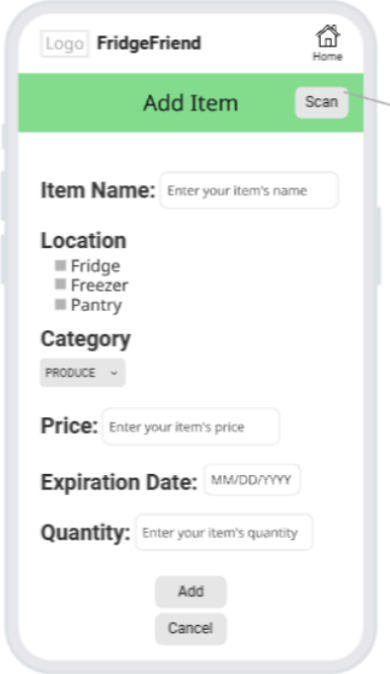

The food inventory interface will allow the user to add items including their location, expiration date, quantity, and have the ability to mark an item as opened. Users will be able click on an item, see the item's specific information and have the ability to edit it. Additionally, users will be able remove an item from the list after it has been consumed.

The expiration calendar interface shows the user a monthly calendar view with the item's name on the calendar date of when it is due to expire. Users can select the food item within the calendar to view all of its information. The user can switch between months with a dropdown to view all items expiring that month.
The expense tracking interface will show users the amount of money that they spent with a breakdown by month. Additionally, users will be able to see how much money they wasted from food that they had to throw away without consuming.

Item Info Screen

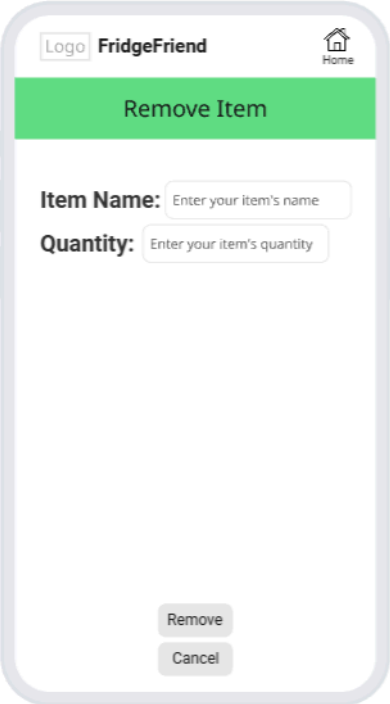



Add Item Screen

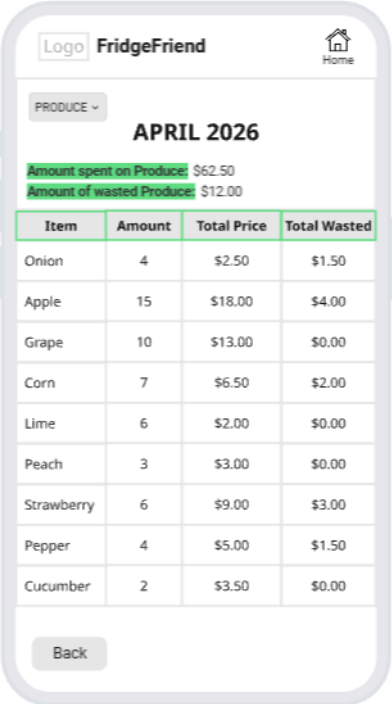



Barcode Scanner Screen

Remove Item Screen

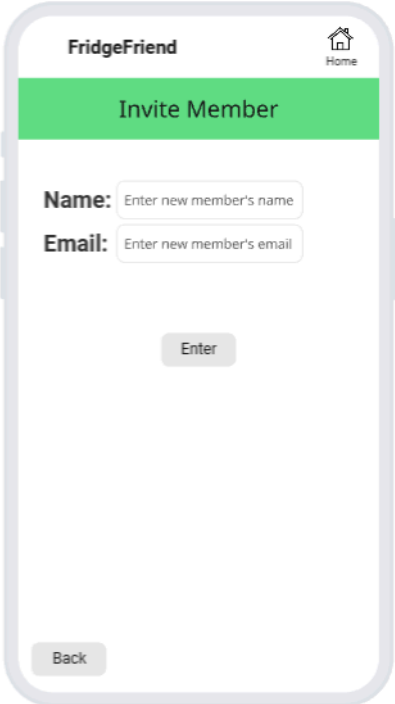



Month Spending Screen




| Item | Amount | Total Price | Total Wasted |
|---|---|---|---|
| Onion | 4 | $2.50 | $1.50 |
| Apple | 15 | $18.00 | $4.00 |
| Grape | 10 | $13.00 | $0.00 |
| Corn | 7 | $6.50 | $2.00 |
| Lime | 6 | $2.00 | $0.00 |
| Peach | 3 | $3.00 | $0.00 |
| Strawberry | 6 | $9.00 | $3.00 |
| Pepper | 4 | $5.00 | $1.50 |
| Cucumber | 2 | $3.50 | $0.00 |

Invite Member Screen




*This section will be updated as more components are added to the application.*

## 3.2 Hardware Interfaces

FridgeFriend requires access to the mobile device's camera to support the barcode scanner functionality for item entry. But due to budget constraints, complete functionality of the barcode scanner won't be possible since it depends on the access to a paid API service. FridgeFriend must be compatible with touch-screen interfaces and notification hardware of when items will expire by storing the timestamp in a database to deliver expiration warnings and alerts.A reliable internet connection will be required for users to log in and sync their food inventory list.

*This section will be updated as more components are added to the application.*

## 3.3 Software Interfaces

FridgeFriend is built with HTML, CSS, TypeScript, and React hooks for an intuitive user interface. The backend is also built with TypeScript and supported by third-party libraries, such as Firebase for database functions.

*This section will be updated as more components are added to the application.*

## 3.4 Communications Interfaces

The system utilizes HTTP protocols for communication between FridgeFriend and the backend server to confirm data integrity during synchronization. FridgeFriend requires a reliable network connection to update the food inventory list across linked household devices to keep an accurate and current list. To protect user information, FridgeFriend will use encryption for account information and notifications.

*This section will be updated as more components are added to the application.*

# 4. System Features

## 4.1 Item Management

### 4.1.1 Description and Priority

Item Management allows the user to maintain a digital inventory of their physical products in their fridge, pantry, and freezer. Allowing the user to add, edit, and delete items to ensure accurate inventory management, High Priority

### 4.1.2 Stimulus/Response Sequences

- User clicks the "Add Item" button → System opens add item window
- User enters item information → System displays the information in the preview

- User scans a product barcode → System identifies the product via API and auto-fills item information
- User toggles “opened” status → System applies freshness logic to the item
- User adjusts item quantity → System updates the numerical count of the item
- User applies a filter → System displays only items matching the criteria (Location, Category)

#### 4.1.3 Functional Requirements

F_REQ-1.1: The system shall allow users to manually add an item.
F_REQ-1.2: The system shall allow users to edit an item’s quantity.
F_REQ-1.3: The system shall allow users to remove an item.
F_REQ-1.4: The system shall allow users to scan a barcode for an item.
F_REQ-1.5: The system shall allow users to mark an item as “opened” to track freshness.
F_REQ-1.6: The system shall allow users to save “Frequently Bought” items to a list.
F_REQ-1.7: The system shall automatically set the date each item was added.
F_REQ-1.8: The system shall display the expiration date of each item.
F_REQ-1.9: The system shall allow users to filter the item list.
F_REQ-1.10: The system shall allow users to select how many days before an item expires to receive a warning.
F_REQ-1.11: The system shall display a warning when items are "near expiring".

## 4.2 Cost Tracking

#### 4.2.1 Description and Priority

Cost Tracking allows the user to view total amount spent over a designated interval (1month/1year) as well as total amount spent on unused food logged into their fridge, pantry and freezer. This provides the user with helpful insight into spending habits, Medium priority.

#### 4.2.2 Stimulus/Response Sequences

- User enters price when they “Add Item” → System saves the cost data to the item info
- User clicks on “Spending” → System displays the total price of all current inventory
- User views “Wasted Food” calculation → System calculates the total price of items removed without being marked as consumed

#### 4.2.3 Functional Requirements

F_REQ-2.1: The system shall allow a user to view the price of an item.
F_REQ-2.2: The system shall allow a user to input the price of an item.
F_REQ-2.3: The system shall calculate the total price of all entered items.
F_REQ-2.4: The system shall display the total price of all entered items.
F_REQ-2.5: The system shall calculate the total price of unused items.
F_REQ-2.6: The system shall allow the user to select a time interval for viewing total spending.

## 4.3 Expiration Calendar

### 4.3.1 Description and Priority

The Expiration Calendar lets the user see the dates all of their items expire on a calendar interface. It will display the date and under it will show which items expire that certain day. The user can click the dates to see more information about the items.

### 4.3.2 Stimulus/Response Sequences

- User navigates to Calendar page → System displays a monthly grid with items that are expiring soon
- User clicks on arrow to change month → System refreshes the grid calendar with items expiring on individual days
- User clicks on a specific day on the calendar → System opens a detailed list of all items expiring on that day
- User clicks on an item → System displays the item information
- User add/remove an item → System automatically updates the markers on the corresponding calendar dates

### 4.3.3 Functional Requirements

F_REQ-3.1: The system shall display a calendar showing all dates within the month.
F_REQ-3.2: The system shall display the items on the calendar under their corresponding expiration date.
F_REQ-3.3: The system shall allow the user to switch between months.
F_REQ-3.4: The system shall allow the user to view individual days displayed on the calendar.
F_REQ-3.5: The system shall display a list of expired food on a selected date.
F_REQ-3.6: The system shall display details about the item when the individual date is clicked.
F_REG-3.7: The system shall display the location, category, and price of an item when the detailed view is shown.
F_REQ-3.8: The system shall update calendar when a new item is added/removed.

## 4.4 Account

### 4.4.1 Description and Priority

This application will require that users create accounts. This will allow for the system to track a users inventory information and share this information with their household members, syncing to keep up to date user data, High Priority

### 4.4.2 Stimulus/Response Sequences

- User clicks “Sign Up” → System prompts user to enter username, email, and password
- User submits registration → System creates account and prompts user to setup 2FA
- User enters login information → System requests a secondary authentication code
- User enters 2FA code → System validates login and directs user to the home screen

### 4.4.3 Functional Requirements

F_REQ-4.1: The system shall encrypt the user information that is given.
F_REQ-4.2: The system shall allow users to sign up with an email and password.
F_REQ-4.3: The system shall allow users to delete their accounts.
F_REQ-4.4: The system shall require a password and a unique identifier for all primary authentication attempts.
F_REQ-4.5: The system shall require two factor authentication to establish a secure session.
F_REQ-4.6: The system shall provide a one-time bypass for two factor authentication during initial account creation.

# 5. Other Nonfunctional Requirements

## 5.1 Performance Requirements

NF_REQ-1.1: The system shall update the item inventory list across linked household devices within 3 seconds of an item being added to the inventory, at least 99% of the time.
NF_REQ-1.2: The system shall parse the data from a barcode scan, displaying the product details within 2 seconds of an image being taken, at least 99% of the time.
NF_REQ-1.3: The system shall display a warning for expiry within one hour, at least 99% of the time.
NF_REQ-1.4: The system shall display search/filter results within 1 second, at least 99% of the time.
NF_REQ-1.5: The system shall calculate the total price spent on all entered items within 4 seconds, at least 99% of the time.
NF_REQ-1.6: The system shall maintain 99.95% uptime, allowing for only 4 hours of downtime per year.

## 5.2 Safety Requirements

NF_REQ-2.1: The system shall include a visible disclaimer stating expiration date alerts are estimates and users should inspect their food before consumption.

## 5.3 Security Requirements

NF_REQ-3.1: The system shall restrict household inventory access to authorized members only.
NF_REQ-3.2: The system shall execute a hard delete of all personally identifiable information from the database within 24 hours.
NF_REQ-3.3:The system shall protect Personally Identifiable Information (contact information, authentication data, and digital identifiers) at rest (AES-256) and in transit (TLS 1.3).

### 5.4 Software Quality Attributes

NF_REQ-4.1: The system shall operate without crashing 99% of the time on 95% of actively used IOS and Android.
NF_REQ-4.2: The system shall display field-specific error messages that identify the exact input error (e.g., “Invalid Date Format”).
NF_REQ-4.3: The system shall highlight the missing fields in red and display a “Required Field” prompt when the user saves an item without a name or location.
NF_REQ-4.4: The system shall load the main page within 3 seconds.
NF_REQ-4.5: The system shall not crash if a user enters incomplete or invalid information.

### 5.5 Business Rules

NF_REQ-5.1: The system shall enforce a database constraint that restricts each unique “Household ID” to a single “Primary Account Holder” role at any given time.
NF_REQ-5.2: The system shall block the creation of an item in inventory if the “Item Name” or “Location” (item storage location) attributes are null or only contain whitespace.

## 6. Other Requirements

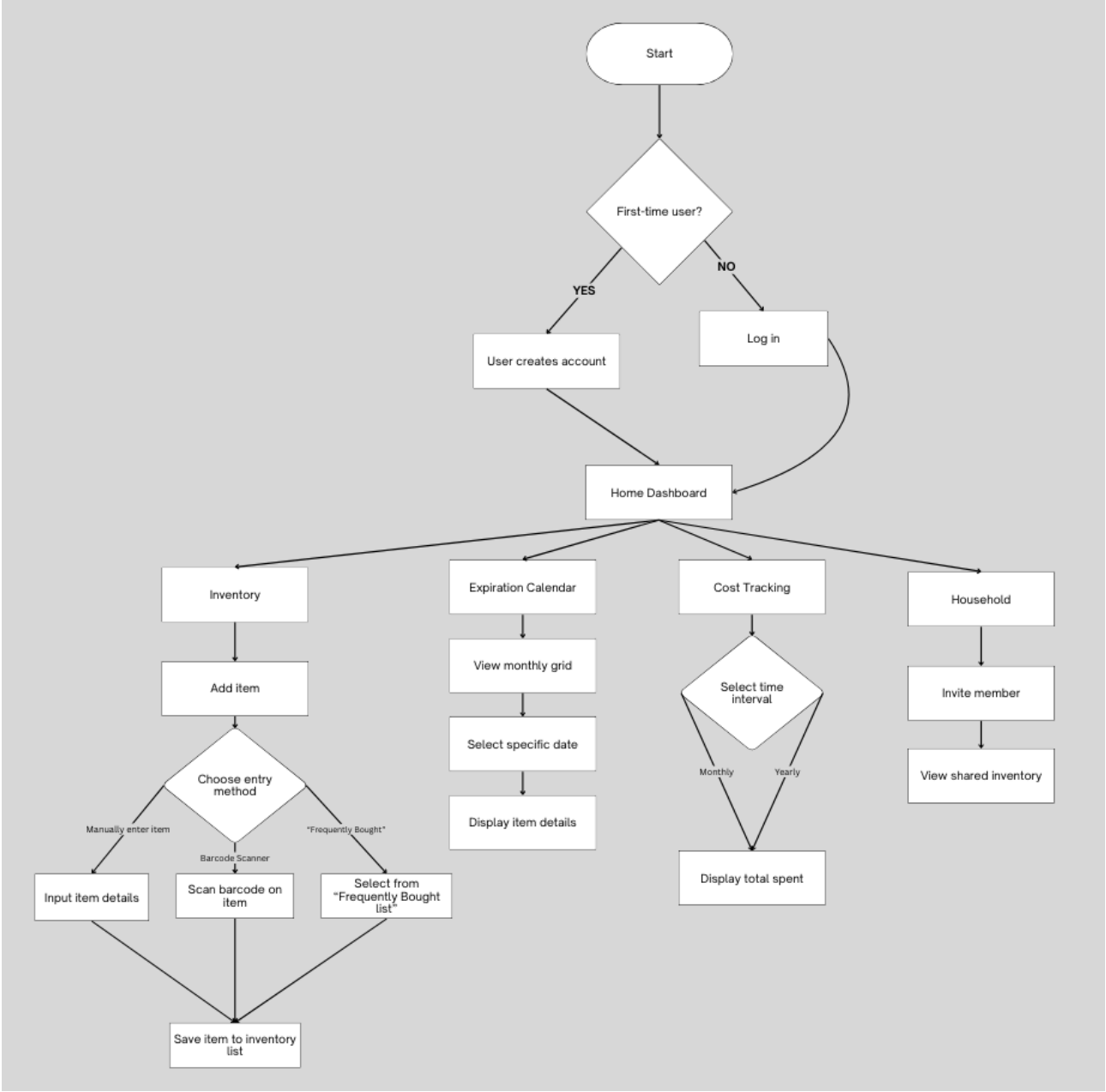
NF_REQ-6.3: The system shall Comply with the California food labeling laws.

## Appendix A: Glossary

Team Development: Team Name
FridgeFriend: App Name
SRS: Software Requirements Specification

## Appendix B: Analysis Models

FridgeFriend Task Flow Diagram




[FridgeFriend Application - Task Flow Diagram]

# Appendix C: To Be Determined List